Resume Matcher Application
Complete User Guide

Version: 1.0 | **Generated:** November 30, 2025 | **Application:** Resume Matcher System

Table of Contents

1. OVERVIEW

The Resume Matcher is a Flask-based web application that facilitates the recruitment process by:

• Allowing recruiters to create job forms and share them with aspirants

• Enabling aspirants to apply by uploading their resumes

• Automatically matching resumes with job descriptions using TF-IDF and cosine similarity

• Ranking candidates based on their resume similarity scores

Key Benefits:

• **For Recruiters**: Easy job form creation, shareable links, automated resume matching

• **For Aspirants**: Simple application process, clear submission confirmation

• **Automated Matching**: AI-powered resume ranking based on job description relevance

2. SYSTEM ARCHITECTURE

Technology Stack:

• **Backend**: Flask (Python web framework)

• **Database**: SQLite3

• **Frontend**: HTML5 with CSS3

• **Matching Algorithm**: TF-IDF Vectorizer + Cosine Similarity (scikit-learn)

• **Server**: Development Flask server (can be replaced with production WSGI server)

Project Structure:

```
web_app/ â”œâ”€â”€ app.py # Main Flask application â”œâ”€â”€ database.py #
Database operations and schema â”œâ”€â”€ matcher_logic.py # Resume matching
algorithm â”œâ”€â”€ verify_db.py # Database verification utility â”œâ”€â”€
web_app/ â”, â”œâ”€â”€ job_descriptions/ # Stores uploaded job description files
â”, â”œâ”€â”€ resumes/ # Stores aspirant resumes (organized by job_id) â”,
â””â”€â”€ match_results.db # SQLite database file â”œâ”€â”€ templates/ â”,
â”œâ”€â”€ recruiter_dashboard.html # Recruiter main dashboard â”, â”œâ”€â”€
upload_job_description.html # Job form creation page â”, â”œâ”€â”€
job_form_details.html # Shareable link display page â”, â”œâ”€â”€
aspirant_form.html # Public application form for aspirants â”, â”œâ”€â”€
submission_success.html # Confirmation page â”, â””â”€â”€ view_results.html #
Results and matching display â””â”€â”€ requirements.txt # Python dependencies
```

3. RECRUITER WORKFLOW

Step 1: Access Recruiter Dashboard

• **URL**: `http://localhost:5000/` or `http://localhost:5000/recruiter`

• **What you see**: List of all created job forms with action buttons

Step 2: Create a New Job Form

• Click **"Create New Job Form"** button

• **URL**: `/upload_job_description`

• Fill in:

• **Job Title**: Name of the position (e.g., "Senior Software Engineer")

• **Your Email Address**: Recruiter's contact email

• **Job Description File**: Upload TXT, PDF, or Word document with job details

• Click **"Create Job Form"**

• System generates a unique Job ID (UUID)

Step 3: Get Shareable Link

• **URL**: `/job_form_details/`

• View displays the public link to share with aspirants

• **Copy to Clipboard** button for easy sharing

• Share link via email, messaging, or social media

Step 4: Monitor Submissions

• Back to Dashboard

• See all job forms in table:

• Job Title

• Job File name

• Recruiter Email

• Status (Active/Inactive)

• Creation Date

• Action Buttons

Step 5: Run Resume Matching

• Once aspirants upload resumes, click **"Run Matching"** button

• System reads all submitted resumes

• Compares each resume with job description

• Calculates similarity scores

• Ranks resumes from highest to lowest match

Step 6: View Results

• Click **"View Results"** button

• See two sections:

1. **Aspirant Submissions**: Table with all applicants

• Applicant Name

• Email Address

• Resume File Name

• Submission Date

2. **Resume Matching Results**: Ranked list

• Rank (1, 2, 3, etc.)

• Resume Filename

• Match Score (0-1, where 1 = perfect match)

• Timestamp of matching

Step 7: Delete Job Form (Optional)

• Click **"Delete"** button on any job form

• Confirmation dialog appears

• Deletes:

• Job description file

• All submitted resumes

• Aspirant submissions database records

• Matching results

• Job metadata

4. ASPIRANT WORKFLOW

Step 1: Receive Application Link

• Recruiter sends shareable link via email or other means

• Link format: `http://your-server/aspirant/`

Step 2: Access Application Form

• **URL**: `/aspirant/`

• Form displays professional interface with instructions

Step 3: Fill Application Form

Required fields:

• **Full Name**: Your complete name

• **Email Address**: Valid email for follow-up

• **Resume File**: Upload PDF, DOC, DOCX, or TXT file

Step 4: Submit Application

• Click **"Submit Application"** button

• File is uploaded and stored

• Submission record saved to database with:

• Aspirant name and email

• Resume filename (with unique identifier)

• Submission timestamp

Step 5: Confirmation

• **URL**: `/submission_success.html`

• Success message displayed

• Confirmation that application was received

• Message about recruiter review process

5. TECHNICAL DETAILS

Database Schema

Table: job_descriptions

```
CREATE TABLE job_descriptions ( job_id TEXT PRIMARY KEY, recruiter_email TEXT NOT NULL, job_title TEXT NOT NULL, original_filename TEXT NOT NULL, upload_timestamp TEXT NOT NULL, status TEXT DEFAULT 'active' )
```

Table: aspirant_submissions

```
CREATE TABLE aspirant_submissions ( submission_id INTEGER PRIMARY KEY AUTOINCREMENT, job_id TEXT NOT NULL, aspirant_name TEXT NOT NULL, aspirant_email TEXT NOT NULL, resume_filename TEXT NOT NULL, submission_timestamp TEXT NOT NULL, FOREIGN KEY (job_id) REFERENCES job_descriptions(job_id) )
```

Table: match_results

```
CREATE TABLE match_results ( id INTEGER PRIMARY KEY AUTOINCREMENT, job_id TEXT NOT NULL, resume_filename TEXT NOT NULL, match_score REAL NOT NULL, timestamp TEXT NOT NULL )
```

Resume Matching Algorithm

Method: TF-IDF (Term Frequency-Inverse Document Frequency) + Cosine Similarity

Process:

1. Convert job description to TF-IDF vector
2. Convert each resume to TF-IDF vector
3. Calculate cosine similarity between job description and each resume
4. Results in scores between 0 and 1:

• 0 = No relevance

• 1 = Perfect match

• 0.5 = Moderate relevance

Why This Works:

• Identifies important keywords in job description

• Finds resumes with similar keyword distribution

• Language-independent (works with any language)

• Fast computation even for large files

File Organization

Job Descriptions:

• Stored in: `web_app/job_descriptions/`

• Filename format: `.txt`

• Original filename preserved in database

Resumes:

• Stored in: `web_app/resumes//`

• Filename format: `__`

• Organized by job_id for easy management

• Multiple resumes per job supported

6. DATABASE STRUCTURE

Data Relationships

job_descriptions (Primary) â†“ â”œâ”€â”€ aspirant_submissions (Foreign Key: job_id) â””â”€â”€ match_results (Foreign Key: job_id)

Data Flow

1. Recruiter Upload â†“ job_descriptions table created â†“ 2. Aspirant Submission â†“ aspirant_submissions table records entry Resume file saved to disk â†“ 3. Matching Process â†“ match_results table populated with scores â†“ 4. Results Display â†“ Recruitment dashboard shows all data

7. FEATURES AND FUNCTIONALITY

Recruiter Features

Aspirant Features

Security Features

• **File Upload Validation**: Only allows specific file types (TXT, PDF, DOC, DOCX)
• **Secure Filenames**: Uses werkzeug.utils.secure_filename()
• **Unique Identifiers**: UUIDs prevent job_id collisions
• **Database Integrity**: Foreign key constraints
• **Max Upload Size**: 16 MB limit per file
• **Error Handling**: Comprehensive error messages

User Experience Features

• **Responsive Design**: Works on desktop and mobile
• **Confirmation Dialogs**: Prevents accidental deletions
• **Professional UI**: Clean, modern interface
• **Clear Instructions**: Helpful hints throughout
• **One-Click Copy**: Easy link sharing
• **Real-time Feedback**: Immediate form submission response
• **Data Validation**: Required field validation on forms

8. API ENDPOINTS

Recruiter Routes

GET / â†’ Redirect to /recruiter GET /recruiter â†’ Recruiter Dashboard GET /upload_job_description â†’ Job creation form POST /upload_job_description â†’ Process job upload GET /job_form_details/ â†’ Show shareable link GET /match_resumes/ â†’ Run matching algorithm GET /view_results/ â†’ Display results GET /view_job_description/ â†’ View job file content POST /delete_job/ â†’ Delete job and data

Aspirant Routes

GET /aspirant/ â†’ Application form POST /aspirant/ â†’ Submit application

9. INSTALLATION & SETUP

Requirements

• Python 3.8+

• Flask

• scikit-learn

• werkzeug

Installation Steps

```
# 1. Navigate to project directory cd c:\Users\rahul\OneDrive\Desktop\web_app # 2. Create virtual environment python -m venv venv # 3. Activate virtual environment venv\Scripts\activate # 4. Install dependencies pip install flask scikit-learn # 5. Initialize database python database.py # 6. Run application python app.py
```

Access Application

• Open browser

• Navigate to: `http://localhost:5000`

• Application runs on port 5000

10. TROUBLESHOOTING

Issue: "No such table: aspirant_submissions"

Solution: Delete database and reinitialize

```
Remove-Item -Path "web_app/match_results.db" -Force python database.py
```

Issue: File upload fails

Solution:

• Check `web_app/job_descriptions/` and `web_app/resumes/` exist

• Verify file size < 16 MB

• Ensure file format is supported (TXT, PDF, DOC, DOCX)

Issue: Matching shows no results

Solution:

• Ensure resumes are uploaded before running matching

• Check that job description file is readable

• Verify resume files are in correct directory

11. FUTURE ENHANCEMENTS

Possible improvements:

- User authentication and roles
- Multiple recruiter accounts
- Email notifications to aspirants
- Resume parsing for structured data
- Advanced filtering and search
- Export results to PDF/Excel
- Bulk operations for multiple jobs
- Resume scoring breakdown/explanation

CONCLUSION

The Resume Matcher application provides a complete, automated solution for recruitment. It simplifies the hiring process by:

1. Making it easy for recruiters to create and share job forms
2. Providing aspirants a simple way to apply
3. Automatically ranking candidates based on resume relevance
4. Centralizing all recruitment data in one place

The application demonstrates modern web development practices with a clean separation of concerns, proper database design, and user-friendly interface.

Version: 1.0

Last Updated: November 30, 2025

Support: Contact application administrator

This document provides a complete guide to the Resume Matcher Application.
For support, contact the application administrator.